

Fake Project Prompt: C ASCII Graphics Editor

Project Overview:

Build a simple console-based C graphics editor that uses a 50x25 ASCII canvas and supports drawing lines. The editor should allow adding, modifying, deleting, and listing objects, and then render the result in text format.

Function Prompts:

1. `clear_canvas`: Implement `clear_canvas(canvas)` to fill the entire canvas with the `EMPTY` character for a given canvas.
2. `set_pixel`: Implement `set_pixel(canvas, x, y)` to safely set a pixel only when coordinates are inside the grid boundaries.
3. `display_canvas`: Implement `display_canvas(canvas)` to print the current canvas rows and columns as ASCII art.
4. `draw_line`: Implement `draw_line(canvas, x1, y1, x2, y2)` using Bresenham-style line drawing to connect two points.
5. `draw_rectangle`: Implement `draw_rectangle(canvas, x1, y1, x2, y2)` to draw the outline of a rectangle using the `draw_line` function.
6. `draw_triangle`: Implement `draw_triangle(canvas, x1, y1, x2, y2, x3, y3)` by drawing lines between three vertices.
7. `draw_circle`: Implement `draw_circle(canvas, xc, yc, radius)` using a midpoint circle algorithm to draw a circle.
8. `draw_object`: Implement `draw_object(canvas, object)` to dispatch drawing logic based on the object type.
9. `redraw_canvas`: Implement `redraw_canvas(canvas, objects, count)` to clear the canvas and redraw everything.
10. `shape_name`: Implement `shape_name(type)` to return a human-readable name for each shape enum value.
11. `list_objects`: Implement `list_objects(objects, count)` to print stored object details and their coordinates.
12. `read_shape_type`: Implement `read_shape_type(type)` to read the object type selection from the user.
13. `read_object`: Implement `read_object(object)` to read shape-specific parameters from the user based on the type.
14. `add_object`: Implement `add_object(objects, count)` to append a new object to the object list after validation.
15. `delete_object`: Implement `delete_object(objects, count)` to remove an object by index and shift the remaining objects.
16. `modify_object`: Implement `modify_object(objects, count)` to update an existing object entry with new user input.
17. `print_menu`: Implement `print_menu()` to show the main command menu options to the user.
18. `main`: Implement `main()` to run the command loop, handle user choices, and maintain the canvas and object list.

Data Structures:

- `ShapeType` enum: Define `LINE`, `RECTANGLE`, `TRIANGLE`, `CIRCLE` values.
- `Object` struct: Store object type, coordinates, and radius for circles.

User Workflow:

1. Start with an empty canvas.
2. Add objects by type and coordinates.
3. Modify or delete objects as needed.
4. Display the canvas to view the current drawing.
5. List objects for reference.